INTEGRATOR WINDUP AND BUMPLESS TRANSFER

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....4
Recommended Citation.....5

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Produce integrator windup on the DC motor by saturating the actuator, and measure its effect on the response.	☐
2	Implement clamping anti-windup in a discrete-time PI controller and compare the response with and without it.	☐
3	Switch between manual and automatic control with and without integrator initialisation, and explain bumpless transfer.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware and algorithm set-up described in the practical exercise, DC Motor Speed Measurement using an Encoder, which details the operation of a DC motor with an encoder and the measurement of motor speed in revolutions per minute (RPM). The set-up is not repeated here.

The exercise also uses the closed-loop model from the practical exercise, DC Motor Speed Control Using a PI Controller, and the on/off switch from the practical exercise, On/Off Power Switch, read as a digital input to select between manual and automatic control.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. Two practical problems of PID control are produced on the hardware and then solved: integrator windup when the actuator saturates, and a sudden jump in the output when switching from manual to automatic control. The exercise supports the ACE-CORE learning guide, Practical Considerations for PID Control.

Background: Integrator Windup

The PWM output can only apply between 0 V and 5 V to the motor. When the controller asks for more than this, the actuator saturates and the motor cannot follow. The error then stays large, and the integral term keeps growing even though it has no further effect. When the setpoint is later reduced, the large integral term must first unwind before the output can fall, so the motor stays fast for much longer than it should.

Clamping anti-windup stops the integration whenever the output is saturated and the error would drive it further into saturation:

$$I[k] = I[k-1] \qquad \text{if the output is saturated and } e\,u_{raw} > 0$$

$$I[k] = I[k-1] + K_i\,T_s\,e[k] \qquad \text{otherwise}$$

MATLAB Function Block: PI controller with clamping anti-windup and integrator initialisation

```
function [u, u_raw] = fcn(r, y, Kp, Ki, Ts, aw, auto, u_man)
% aw: 1 enables clamping anti-windup; auto: 1 for automatic, 0 for manual
% u: voltage to the motor [V], limited to 0-5 V; u_raw: before the limit
persistent I
if isempty(I), I = 0; end
e = r - y;
if ~auto
    I = u_man - Kp*e;           % track the manual output (bumpless)
    u = u_man;  u_raw = u_man;
    return
end
u_raw = Kp*e + I;
u = min(max(u_raw, 0), 5);
saturated = (u ~= u_raw) && (e*u_raw > 0);
if ~(aw && saturated)
    I = I + Ki*Ts*e;            % integrate unless clamped
end
```

Produce and Remove Integrator Windup

1. Replace the PI controller in the speed control model with the MATLAB Function block above, followed by a Gain of 255/5 to the PWM output. Use K_p = 0.05 V/RPM, K_i = 0.33 V/(RPM·s) and T_s = 0.01 s, with *auto* = 1.
2. Log *u_raw*, the second output of the function, which is the controller output before the 0–5 V limit.
3. With *aw* = 0, apply a setpoint of 160 RPM for 5 s, which is above the top speed of the motor, and then step it down to 70 RPM. Log the setpoint, speed, *u_raw* and *u*.
4. Repeat with *aw* = 1.
5. For each run, measure how long after the step down the speed takes to settle within ±2% of 70 RPM, and record it in Table 2.

Example results are shown in Figure 1. They are simulated with the motor model identified in the practical exercise, Parameter Estimation of a DC Motor using

Linear Least Squares (K = 28 RPM/V, τ = 0.15 s), so they show the behaviour to expect rather than measured data. Without anti-windup, *u_raw* climbs to over 40 V while the motor is saturated at about 140 RPM. After the step down, the speed stays at full speed for about 1.5 s until the integral term unwinds. With clamping, *u_raw* stays near the limit and the speed falls almost immediately.

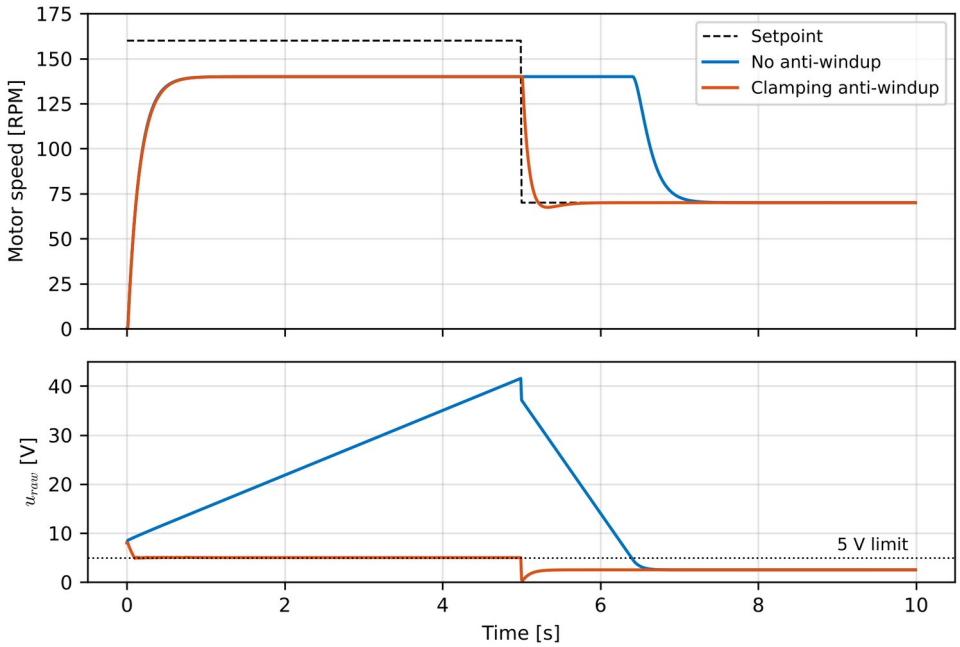


Figure 1: *Simulated Example of Speed (Top) and Unsaturated Controller Output (Bottom), with and without Anti-Windup*

Table 2: *Integrator Windup Results for Your Motor*

Anti-windup	Largest *u_raw* (V)	Speed during saturation (RPM)	Settling time after step down (s)
Off			
Clamping			

Manual to Automatic Transfer

In manual mode, an operator sets the actuator directly. When switching to automatic, the controller output should start from the value the operator had set. Otherwise the output jumps, and so does the process. In the function above, the integral term tracks the manual output while in manual mode, so the controller output equals *u_man* at the moment of switching.

1. Connect the on/off switch to the *auto* input through a Digital Input block. Set *u_man* = 2.5 V and the setpoint to the speed the motor reaches at 2.5 V.
2. Temporarily delete the line `I = u_man - Kp*e;` so the integrator is not initialised. Start in manual, let the speed settle, then switch to automatic. Record the voltage just before and just after switching, and the largest change in speed.
3. Restore the line and repeat the test.
4. Record both results in Table 3.

Example results are shown in Figure 2. They are simulated with the motor model identified in the practical exercise, Parameter Estimation of a DC Motor using Linear Least Squares (K = 28 RPM/V, τ = 0.15 s), so they show the behaviour to expect rather than measured data. Without initialisation, the output drops to almost zero at the switch, and the speed falls sharply before the integral term rebuilds. With initialisation, the switch is not visible in the response.

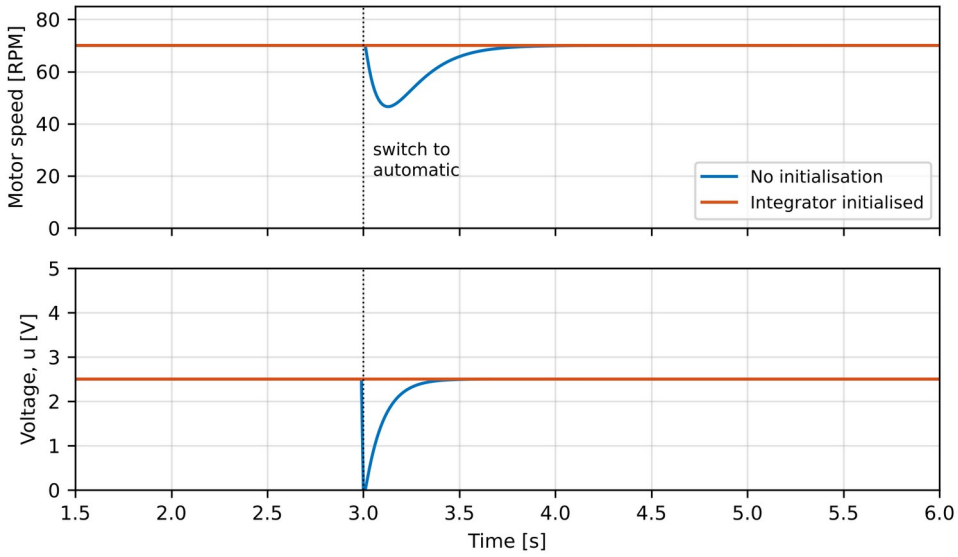


Figure 2: *Simulated Example of Switching from Manual to Automatic at 3 s, with and without Integrator Initialisation*

Table 3: *Manual to Automatic Transfer Results for Your Motor*

Integrator	Voltage before switch (V)	Voltage after switch (V)	Largest change in speed (RPM)
Not initialised	2.5		
Initialised	2.5		

Explain your results, and name one industrial process where a jump in the actuator at the moment of switching could be dangerous.

4.0 Concluding Remarks

This practical exercise investigated two practical issues that shape the performance of PID control on real hardware. Using MATLAB, Simulink and ACE-Lab hardware, integrator windup was produced by commanding a speed beyond the reach of the motor, and clamping anti-windup was shown to remove the long delay it caused.

Switching from manual to automatic control showed that the state of the integrator must match the actuator at the moment of transfer. Initialising the integrator from the manual output gave a bumpless transfer. Both issues arise because a real actuator has limits and a real controller has memory, neither of which appear in an ideal linear design.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Integrator Windup and Bumpless Transfer. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://zenodo.org/records/23725314